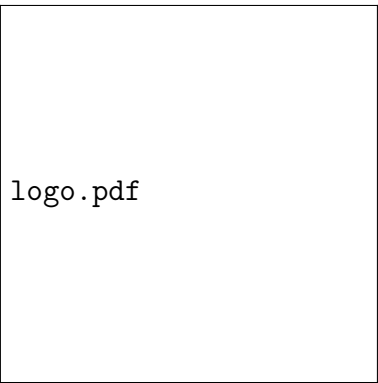


FPGA-Accelerated Network IDS with RISC-V Co-Processor Integration



logo.pdf

Youcef Slimani
Salah Eddine Laifa

Institute of Electrical and Electronic Engineering
Fundamental Teaching Department
M'Hamed Bougara University of Boumerdes

Submitted in partial satisfaction of the requirements for the
Degree of Bachelor
in Electrical and Electronic Engineering

Supervisor Dr. Touzzout Walid

May 2026

Dedications

This thesis is dedicated to our families for their endless support and encouragement throughout our studies.

We further dedicate this work to all our colleagues and teachers who contributed to our academic journey.

Acknowledgements

We would like to express our deepest gratitude to our supervisor, Dr. Touzzout Walid, for his invaluable guidance, patience, and constant encouragement throughout this project.

We would also like to thank the IGEE (ex. INELEC) department for providing the necessary facilities and resources.

Abstract

This thesis presents a hardware-based Network Intrusion Detection System (IDS) implemented on the programmable logic of a Zynq-7020 FPGA, integrated with a RISC-V co-processor developed by a parallel project. The IDS inspects every incoming Ethernet frame at wire speed, extracting header fields from the Ethernet, IPv4, TCP, and UDP layers and evaluating them against a set of twenty detection rules. All rule evaluations complete in a single clock cycle, giving the system a deterministic, constant-latency response to every packet regardless of traffic volume.

The hardware design is described in VHDL and verified through behavioural simulation in Vivado. The integration architecture couples the IDS pipeline to the BigDCore RISC-V processor through a memory-mapped shared register bank: the IDS continuously writes threat data and alert flags, while the RISC-V reads these values to decide alert severity, update detection thresholds at runtime, and transmit formatted alerts over UART.

Keywords: FPGA, VHDL, Intrusion Detection System, Zynq-7020, RISC-V, network security, hardware acceleration, Vivado.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Network traffic volumes and attack sophistication have grown to a point where software intrusion detection systems face fundamental throughput limitations. A software IDS running on a general-purpose CPU must copy packet data through the OS network stack, schedule the inspection process, and share the CPU with other tasks—all of which introduce variable latency and reduce the fraction of packets that can actually be inspected at high line rates.

Reconfigurable hardware offers a different trade-off: inspection logic is implemented directly as digital circuits in an FPGA, so packets are examined as bytes arrive, with no operating system, no scheduling overhead, and no data copies. The cost is development complexity; the benefit is deterministic, wire-speed inspection.

This project builds a hardware IDS in VHDL, targeting the MicroPhase Z7-Lite development board (Zynq-7020, XC7Z020-1CLG400C), and integrates it with a RISC-V processor built independently by a collaborator. The Zynq device is a natural fit: its ARM Cortex-A9 Processing System (PS) receives Ethernet packets through the on-chip GigE MAC and can forward raw frame data to the Programmable Logic (PL) fabric via AXI-Stream DMA, where the IDS pipeline runs. The RISC-V processor also resides in the PL fabric and communicates with the IDS through a shared memory-mapped register bank.

The IDS is developed and validated entirely through behavioural simulation before any hardware integration, allowing each layer of the protocol stack to be verified independently with injected test packets.

1.1 Project Scope

The IDS component of this project is responsible for:

- Receiving a raw byte stream representing Ethernet frames.
- Parsing Ethernet, IPv4, TCP, and UDP headers.
- Evaluating twenty detection rules spanning four protocol layers.
- Accumulating a weighted threat score and classifying traffic severity.
- Exposing all status data and configurable rule parameters to the RISC-V co-processor through a shared register bank.

Hardware integration (block design, AXI-Stream DMA, PS bare-metal driver) constitutes the deployment path and is described in Chapter ??.

1.2 Document Structure

Chapter ?? describes the IDS pipeline architecture and the design of each VHDL module. Chapter ?? lists all twenty detection rules and the reasoning behind each one. Chapter ?? presents the simulation methodology and results. Chapter ?? covers the integration architecture with the RISC-V co-processor and the hardware deployment plan. Chapter ?? concludes with a summary of achievements and directions for future work.

Chapter 2

IDS Pipeline Architecture

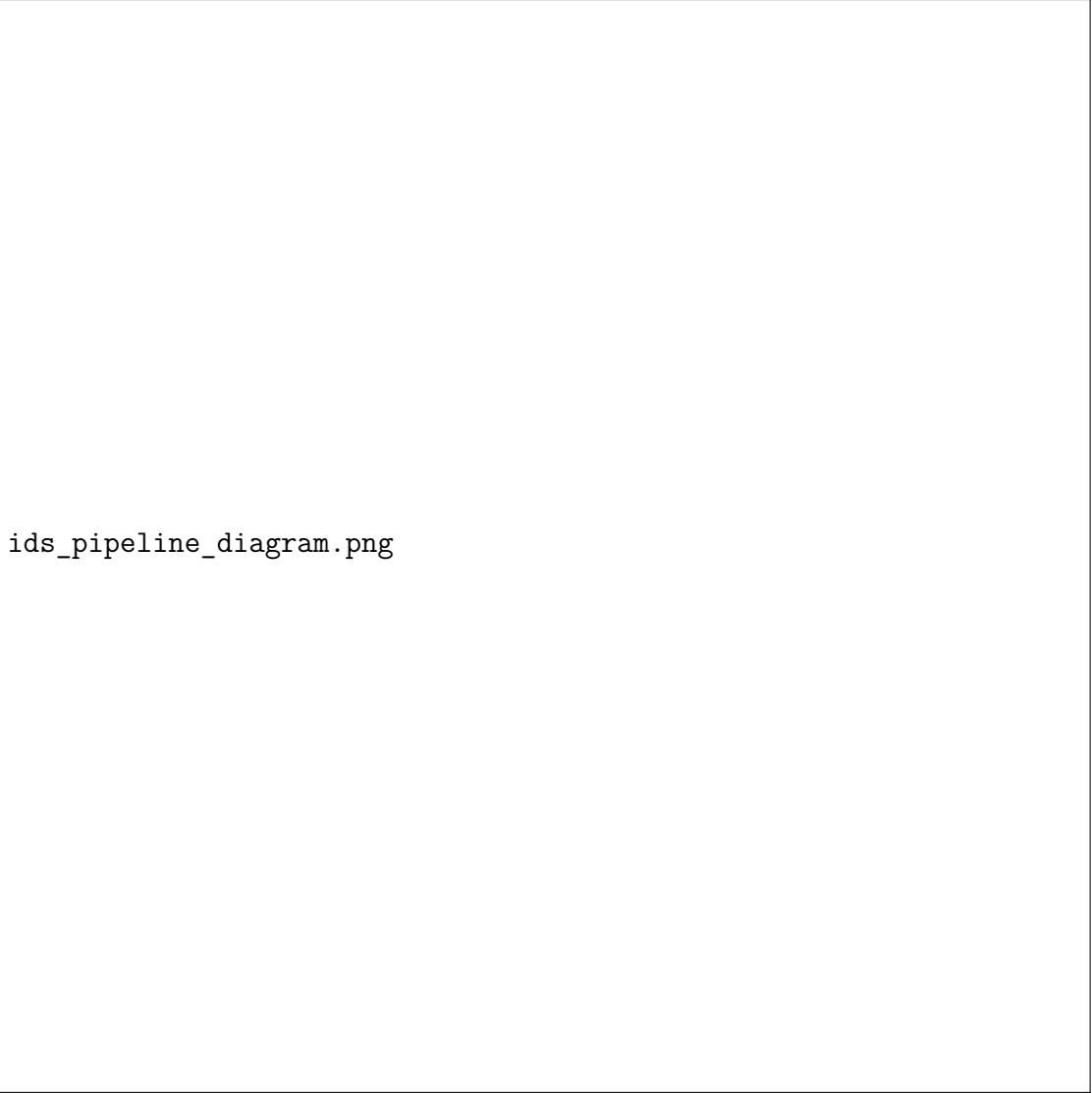
2.1 Design Philosophy

The IDS is structured as a linear pipeline of independent modules, each consuming the same byte stream that flows from a physical interface receiver. Parsers at different protocol layers operate simultaneously on the same data; rule checkers fire one clock cycle after their corresponding parser completes. This means the system does not buffer a full packet before beginning inspection—field extraction and rule evaluation are interleaved with frame reception.

The choice of a byte-stream interface as the internal contract between modules makes each module independently testable and allows the physical reception mechanism to be swapped (from RMI simulation to AXI-Stream DMA for hardware) without modifying any parser or checker.

2.2 Top-Level Structure

The pipeline is wired together in a single top-level entity, `ids_top.vhd`. It instantiates all sub-modules and connects them through internal signals. The byte stream—`rx_byte`, `rx_valid`, `rx_sof`, `rx_eof`—is broadcast to every parser simultaneously.



ids_pipeline_diagram.png

Figure 2.1: IDS pipeline architecture showing parallel parsers and rule engines.

The four-signal byte-stream interface carries the following semantics:

- `rx_sof` pulses for one clock cycle at the start of a new frame.
- `rx_byte` / `rx_valid` delivers one byte per clock when valid.
- `rx_eof` pulses for one clock cycle at the end of the frame.

This interface is the internal standard across all modules.

2.3 RMII Receiver (`rmii_rx.vhd`)

In simulation, Ethernet frames are injected using RMII signalling: two data bits (a *dibit*) are presented every clock cycle at 50 MHz, LSB-first. The RMII receiver is a three-state FSM (IDLE, PREAMBLE, DATA) that assembles four consecutive dibits into one byte using a right-shift register:

```

1 shift_reg <= rmii_rxd & shift_reg(7 downto 2);
2 if dibit_cnt = "11" then
3     rx_byte <= rmii_rxd & shift_reg(7 downto 2);
4     rx_valid <= '1';
5 end if;

```

Listing 2.1: Dibit assembly in `rmii_rx.vhd`

The preamble state counts at least eight 01 dibits before accepting the Start Frame Delimiter (11). On hardware, this module is replaced by an AXI-Stream receiver that accepts bytes directly from the PS DMA engine; the rest of the pipeline is unchanged.

2.4 Ethernet Parser (`eth_parser.vhd`)

The Ethernet parser extracts the destination MAC, source MAC, and EtherType from the first 14 bytes of each frame. It uses a sequential FSM with states for each header field group (DST_MAC, SRC_MAC, ETHERTYPE, PAYLOAD, DONE). On the DONE state, it asserts `frame_done` and one of `is_ipv4`, `is_arp`, or `is_unknown` based on the EtherType value (0x0800, 0x0806, or other).

A key design decision is that all output signals default to '0' at the top of the clocked process and are only overridden in the DONE state. This prevents latches and ensures outputs are valid for exactly one clock cycle.

2.5 Frame Counter (`frame_counter.vhd`)

The frame counter accumulates per-second frame statistics (total, IPv4, ARP, unknown) using a hardware divider: a counter incremented every clock cycle triggers a `one_sec_tick` when it reaches `CLK_FREQ_HZ - 1`. On that tick, the running accumulators are latched to output registers and reset. This value is passed as a generic so the same code simulates at `CLK_FREQ_HZ = 1000` (1 kHz) and synthesizes at `CLK_FREQ_HZ = 50 000 000` (50 MHz), making testbenches fast without any code changes.

2.6 IPv4 Parser (`ipv4_parser.vhd`)

The IPv4 parser skips the 14-byte Ethernet header (counted in the `ETH_HEADER` state) and then extracts the following fields from the IP header:

Table 2.1: IPv4 header fields extracted by `ipv4_parser.vhd`

Field	Byte offset	Width
Version / IHL	0	4 + 4 bits
Total Length	2–3	16 bits
Flags / Fragment Offset	6–7	3 + 13 bits
TTL	8	8 bits
Protocol	9	8 bits
Source IP	12–15	32 bits
Destination IP	16–19	32 bits

The IHL field determines where the IP header ends ($\text{IHL} \times 4$ bytes). The parser computes this boundary dynamically and transitions to `WAIT_EOF` once all header bytes have been consumed, asserting `parse_done` and `parse_valid` (the latter only when `ip_version = 4`) after the frame ends.

2.7 IPv4 Checker (`ipv4_checker.vhd`)

The IPv4 checker receives the parsed fields when `parse_done` is asserted and evaluates all rules combinatorially inside a clocked process in a single clock cycle. A local variable `any_alert` is OR'd across all rule conditions and drives the `alert_any` output. Individual alert signals (`alert_bad_version`, `alert_fragment`, etc.) are also driven, allowing the scoring engine to weight them differently.

2.8 TCP Parser (`tcp_parser.vhd`)

The TCP parser shares the same byte stream as the IPv4 parser. Because the TCP header begins at a byte offset that depends on the IP IHL field, the parser captures `ip_ihl` and `ip_protocol` from the IPv4 parser's output signals before the frame starts. It then skips to byte $14 + \text{IHL} \times 4$ (Ethernet header + IP header) before beginning TCP field extraction.

Fields extracted: source port, destination port, sequence number, and the flags byte. The flags byte at offset 13 of the TCP header encodes FIN, SYN, RST, PSH, ACK, and URG in bits 0–5.

2.9 TCP Checker (`tcp_checker.vhd`)

The TCP checker uses VHDL `alias` declarations to give readable names to individual flag bits:

```

1 alias flag_fin : std_logic is tcp_flags(0);
2 alias flag_syn : std_logic is tcp_flags(1);
3 alias flag_rst : std_logic is tcp_flags(2);

```

Listing 2.2: Flag aliases in `tcp_checker.vhd`

Rules are expressed directly as Boolean conditions on these aliases, making the intent clear without bit-masking arithmetic.

2.10 UDP Parser (`udp_parser.vhd`)

The UDP header is 8 bytes long: source port (2), destination port (2), length (2), and checksum (2). The parser reuses the same skip-to-transport-layer logic as the TCP parser, activated only when `ip_protocol = 0x11`. The `parse_valid` output is asserted only for UDP packets.

2.11 UDP Checker (`udp_checker.vhd`)

The UDP checker evaluates nine rules on the extracted UDP fields and a configurable set of forbidden port values, shared with the TCP checker through the rule register bank.

Chapter 3

Detection Rules

3.1 Overview

The IDS evaluates twenty detection rules across four protocol layers. Rules are grouped by the checker module that evaluates them. Each rule fires for exactly one clock cycle when its condition is met; the alert is latched and its weight added to the threat score.

3.2 IPv4 Rules (R1–R12)

3.2.1 Rules R1–R7: Core Validity Checks

These seven rules detect structurally malformed or trivially suspicious IPv4 packets.

Table 3.1: IPv4 detection rules R1–R7

Rule	Condition	Attack / Anomaly
R1	<code>ip_version</code> $\neq$ 4	Non-IPv4 / malformed packet
R2	<code>ip_ihl</code> $<$ 5	Invalid header length
R3	<code>ip_length</code> $<$ 20	Impossible packet size
R4	<code>ip_ttl</code> $\leq$ 1	Suspicious TTL / traceroute
R5	<code>ip_frag_off</code> $>$ 0	Fragmentation attack
R6	<code>ip_src</code> = 0.0.0.0	Spoofed source address
R7	<code>ip_src</code> = <code>ip_dst</code>	Land attack

3.2.2 Rules R8–R12: Bogon and Header Abuse

These five rules detect packets whose source addresses belong to reserved or unroutable address blocks, or whose header fields abuse optional mechanisms.

Table 3.2: IPv4 detection rules R8–R12

Rule	Condition	Attack / Anomaly
R8	<code>ip_src[31:24] = 0x7F</code>	Loopback source (127.x.x.x)
R9	<code>ip_src[31:16] = 0xA9FE</code>	Link-local source (169.254.x.x)
R10	<code>ip_src[31:28] = 0xE</code>	Multicast source (224–239.x.x.x)
R11	<code>ip_ihl > 5</code>	IP options present
R12	<code>ip_flags[2] = '1'</code>	Reserved IPv4 flag bit set

Rules R8–R10 share a single enable bit (`bogon_check_en`) in the rule register bank so the RISC-V can disable them all at once on a network segment where such addresses may legitimately appear.

3.3 TCP Rules (T1–T5)

Table 3.3: TCP detection rules T1–T5

Rule	Condition	Attack / Anomaly
T1	<code>SYN = 1 and FIN = 1</code>	Illegal flag combination
T2	<code>SYN = 1 and RST = 1</code>	Illegal flag combination
T3	<code>All flags = 0x00</code>	NULL scan (port probe)
T4	<code>All flags = 0xFF</code>	XMAS scan (port probe)
T5	<code>dst_port ∈ forbidden set</code>	Access to forbidden service

The forbidden port set (rule T5) is stored in configurable registers and defaults to Telnet (23), FTP (21), and TFTP (69). The RISC-V processor can update these values without reprogramming the FPGA.

3.4 UDP Rules (U1–U9)

Table 3.4: UDP detection rules U1–U9

Rule	Condition	Attack / Anomaly
U1	<code>udp_dst_port</code> $\in$ forbidden set	Access to forbidden UDP service
U2	<code>udp_length</code> < 8	Invalid UDP length field
U3	<code>udp_length</code> $= 0$	Completely malformed UDP packet
U4	<code>udp_src_port</code> $\in \{53, 123, 11211, 19\}$	Amplification attack response (DNS/NTP/Memcached/CHARGEN)
U5	<code>ip_src</code> $=$ <code>ip_dst</code> $\wedge$ <code>udp_src_port</code> $=$ <code>udp_dst_port</code>	UDP land attack
U6	<code>udp_length</code> > 1472	Oversized datagram (exceeds Ethernet MTU)
U7	<code>udp_src_port</code> $= 0 \vee$ <code>udp_dst_port</code> $= 0$	Reserved port zero used
U8	<code>udp_dst_port</code> $= 1900$	SSDP/UPnP abuse
U9	<code>udp_dst_port</code> $= 11211$	Memcached amplification request

3.5 Alert Weighting

Each alert type is assigned a weight, and the scoring engine accumulates weights additively. The RISC-V classifies traffic into three severity tiers based on the accumulated score.

Table 3.5: Alert weights used by the scoring engine

Alert	Rule	Weight
Bad version	R1	5
Bad IHL	R2	5
Bad length	R3	10
Low TTL	R4	5
Fragmentation	R5	15
Spoofed source	R6	20
Land attack	R7	25
Bogon source	R8–R10	15
IP options	R11	10
Reserved flag	R12	10
SYN+FIN	T1	25
SYN+RST	T2	20
NULL scan	T3	20
XMAS scan	T4	20
Forbidden TCP port	T5	15
Forbidden UDP port	U1	15
Malformed UDP length	U2, U3	10
Amplification resp.	U4	20
UDP land attack	U5	25
Oversized UDP	U6	10
Port zero	U7	10
SSDP/UPnP abuse	U8	15
Memcached amplif.	U9	20

The score decays by a configurable rate every second when no new alerts fire, preventing a historical event from permanently elevating the severity level.

Chapter 4

Simulation and Verification

4.1 Methodology

Each checker module is verified independently before the full-pipeline testbench is run. The testbench for a given checker instantiates only the relevant parser-checker pair, injects synthetic byte streams that exercise boundary conditions, and verifies alert signals in the simulation waveform. This makes debugging straightforward: a failing assertion on `alert_syn_fin` in `tcp_checker_tb` is isolated from anything happening in the Ethernet or IP layers.

The full-pipeline testbench (`ids_top_tb.vhd`) then exercises the entire chain from RMII dibit injection to alert latch, confirming that parser outputs are correctly connected as checker inputs and that the alert latch persists across frames.

A critical testbench optimisation is the use of a reduced clock frequency generic:

```
1 u_dut : entity work.ids_top
2     generic map (CLK_FREQ_HZ => 1000)
3     port map (...);
```

Listing 4.1: Generic override in the testbench

At `CLK_FREQ_HZ = 1000`, the frame counter's one-second tick fires after 1000 clock cycles instead of 50 000 000. A 50 MHz simulation would require running tens of millions of cycles to observe counter behaviour; at 1 kHz it is observable in microseconds.

4.2 Test Procedures

The testbench uses VHDL procedures to encapsulate frame injection. Each procedure drives the RMII signals cycle-by-cycle in the correct dibit order, including preamble, SFD, header bytes, and end-of-frame signalling. This allows a complete test case to be expressed in a few lines:

```
1 send_ipv4_frame(
2     src_ip    => x"C0A80101",    -- 192.168.1.1
3     dst_ip    => x"C0A80101",    -- same as source
4     protocol  => x"06",          -- TCP
5     ttl       => x"40"
6 );
7 wait until rising_edge(clk);
8 assert alert_land_attack = '1'
```

```
9 | report "Land_attack_not_detected" severity failure;
```

Listing 4.2: Injecting a land attack in the testbench

4.3 IPv4 Checker

All twelve R1–R12 rules were verified in a single simulation run by injecting a sequence of frames, each crafted to trigger one specific rule while keeping all other fields valid. The rules are grouped into three categories: header validation (R1–R7), bogon address checks (R8–R10), and header abuse detection (R11–R12).

Header Validation Rules (R1–R7)

The following rules were confirmed to fire at the correct simulation time:

- R1: Frame with `ip_version = 6` — `alert_bad_version` asserted.
- R2: Frame with `ip_ihl = 3` — `alert_bad_ihl` asserted.
- R3: Frame with `ip_length = 10` — `alert_bad_length` asserted.
- R4: Frame with `ip_ttl = 1` — `alert_low_ttl` asserted.
- R5: Frame with non-zero fragment offset — `alert_fragment` asserted.
- R6: Frame with `ip_src = 0.0.0.0` — `alert_spoof_src` asserted.
- R7: Frame with `ip_src = ip_dst` — `alert_land_attack` asserted.

Bogon Address Rules (R8–R10)

Bogon checks operate on the upper bits of `ip_src` and fire as soon as the source address bytes are assembled, before the end of the frame is reached. Each rule was verified with a dedicated frame:

- R8: Frame with `ip_src = 127.0.0.1` (`ip_src[31:24] = 0x7F`) — `alert_bogon_loop` asserted.
- R9: Frame with `ip_src = 169.254.0.1` (`ip_src[31:16] = 0xA9FE`) — `alert_bogon_link` asserted.
- R10: Frame with `ip_src = 224.0.0.1` (`ip_src[31:28] = 0xE`) — `alert_bogon_mcast` asserted.

Header Abuse Rules (R11–R12)

- R11: Frame with `ip_ihl = 6` (options field present, header exceeds minimum 20 bytes) — `alert_ip_options` asserted.
- R12: Frame with the reserved IPv4 flag bit set (`ip_flags[2] = '1'`) — `alert_reserved_bit` asserted.

In all twelve cases, no secondary alert was triggered on frames designed to exercise a single rule, confirming that the rules are mutually independent in their evaluation logic.

4.4 TCP Checker Verification

Rules T1–T5 were verified by injecting five TCP frames with specific flag values and destination ports. Key test cases:

- T1: `tcp_flags = 0x03` (SYN+FIN) — `alert_syn_fin` asserted.
- T2: `tcp_flags = 0x06` (SYN+RST) — `alert_syn_rst` asserted.
- T3: `tcp_flags = 0x00` (NULL) — `alert_null_scan` asserted.
- T4: `tcp_flags = 0xFF` (all flags) — `alert_xmas_scan` asserted.
- T5: `dst_port = 23` (Telnet) — `alert_forbidden` asserted.



tcp_checker_waveform.png

Figure 4.1: Simulation waveform showing TCP rules T1–T5 firing in sequence.

4.5 UDP Checker

All nine U1–U9 rules were verified by injecting dedicated UDP frames, each violating exactly one rule while keeping all other fields valid. The rules are grouped into three categories: forbidden port and structural checks (U1–U3), amplification and spoofing detection (U4–U5), and size and service abuse checks (U6–U9).

Forbidden Port and Structural Rules (U1–U3)

- U1: Frame with `udp_dst_port = 69` (TFTP, in the configured forbidden port table) — `alert_udp_forbidden` asserted.
- U2: Frame with `udp_length = 4` (below the minimum valid value of 8) — `alert_udp_short` asserted.

- U3: Frame with `udp_length = 0` — `alert_udp_zero` asserted.

Amplification and Spoofing Rules (U4–U5)

- U4: Frame with `udp_src_port = 53` (DNS response arriving from an amplification reflector) — `alert_udp_amplify` asserted.
- U5: Frame with `ip_src = ip_dst` and `udp_src_port = udp_dst_port` (UDP land attack, both IP-layer and port-layer conditions required simultaneously) — `alert_udp_land` asserted. A frame satisfying only one of the two conditions was also sent and confirmed to produce no alert, verifying the AND logic of the checker.

Size and Service Abuse Rules (U6–U9)

- U6: Frame with `udp_length = 1480` (exceeding the 1472-byte MTU threshold) — `alert_udp_oversize` asserted.
- U7: Frame with `udp_dst_port = 0` (reserved port zero, never valid in legitimate traffic) — `alert_udp_port_zero` asserted.
- U8: Frame with `udp_dst_port = 1900` (SSDP/UPnP, a common amplification vector) — `alert_ssdp_abuse` asserted.
- U9: Frame with `udp_dst_port = 11211` (Memcached amplification request) — `alert_memcached` asserted.

In all nine cases, the alert signal was asserted within the expected number of clock cycles from the relevant header field being assembled, and no secondary alerts were triggered on single-violation frames.

Chapter 5

System Integration Architecture

5.1 Hardware Platform

The MicroPhase Z7-Lite board combines an ARM Cortex-A9 Processing System and Artix-7-equivalent Programmable Logic in a single Zynq-7020 SoC. The on-board Ethernet PHY (RTL8201F) connects to the PS via RGMII, meaning the MAC and PHY interface are handled entirely within the PS without consuming any PL I/O resources. This is the key constraint that shapes the integration architecture: the PL cannot directly observe the raw Ethernet wires, so raw packet bytes must be passed from PS to PL.

Table 5.1: Z7-Lite pin assignments used by the PL design

Signal	Pin	Function
PL_CLK_50M	N18	50 MHz main clock for PL logic
PL_LED1	P15	<code>led_alert</code> output
GPI01_17N	U12	<code>led_active</code> output
PL_KEY1	P16	Active-low reset input

5.2 PS-to-PL Packet Forwarding

The chosen integration path is:

1. The PS ARM processor receives complete Ethernet frames through the GigE MAC using lwIP or a bare-metal DMA driver.
2. The PS strips the 4-byte FCS and writes the remaining frame bytes into a DMA buffer.
3. The AXI-Stream DMA IP transfers the buffer to PL fabric, driving an AXI-Stream slave interface.
4. An `axis_rx.vhd` wrapper converts the AXI-Stream handshake signals (`tvalid`, `tready`, `tdata`, `tlast`) into the four-signal byte-stream interface used internally by all IDS parsers.

This approach keeps the entire parser and checker stack unchanged for hardware use. Only the physical reception module is swapped.



Figure 5.1: PS-PL co-design architecture: packet path from RTL8201F PHY through AXI-Stream DMA to the IDS pipeline, and the shared register bank connecting the IDS to the BigDCore RISC-V.

5.3 AXI-Stream Wrapper (`axis_rx.vhd`)

The AXI-Stream protocol requires a `tready` handshake from the receiver. The wrapper asserts `tready` permanently (the IDS pipeline never stalls the input stream) and maps the remaining signals directly:

- `tvalid` → `rx_valid`
- `tdata[7:0]` → `rx_byte`
- `tlast` → `rx_eof`
- A one-cycle delayed `tvalid` rising edge → `rx_sof`

5.4 RISC-V Co-Processor Integration

5.4.1 BigDCore Memory Interface

The RISC-V processor produced by the collaborating project is named BigDCore. It implements the RV32I base integer ISA as a pipelined 32-bit processor and exposes a simple external memory interface: `DataAdr[31:0]`, `ReadData[31:0]`, `WriteData[31:0]`, `MemWrite`, `ByteEn[1:0]`, and `Mode[1:0]`. Load and store instructions drive this interface directly, so the IDS registers appear to the RISC-V as ordinary memory locations.

5.4.2 Shared Register Bank

The shared register bank (`shared_regs.vhd`) implements 29 32-bit registers at addresses 0x400–0x4B4 in the RISC-V address space. Registers are divided into two groups:

- **Status registers (0x400–0x454):** Written by the IDS hardware, read by the RISC-V. These include the threat score, a 12-bit alert flags bitmask, cumulative alert counts, per-protocol frame counts, frames-per-second statistics, and the fields of the last alerted packet (source/destination IP, ports, TCP flags, TTL, protocol, length).
- **Rule registers (0x480–0x4B4):** Written by the RISC-V, read by the IDS hardware. These include the TTL threshold, fragmentation check enable, scoring thresholds, score decay rate, an alert clear command, and eight configurable forbidden port values.

Table 5.2: Shared register map (selected registers)

Address	Register	Direction
0x400	<code>threat_score</code>	IDS → RISC-V
0x404	<code>alert_flags</code>	IDS → RISC-V
0x408	<code>alert_count_total</code>	IDS → RISC-V
0x428	<code>fps_total</code>	IDS → RISC-V
0x434	<code>last_alert_src_ip</code>	IDS → RISC-V
0x454	<code>threat_level</code>	IDS → RISC-V
0x480	<code>ttl_threshold</code>	RISC-V → IDS
0x488	<code>score_high_threshold</code>	RISC-V → IDS
0x490	<code>score_decay_rate</code>	RISC-V → IDS
0x494	<code>alert_clear</code>	RISC-V → IDS
0x498	<code>forbidden_port_0</code>	RISC-V → IDS
0x800	<code>uart_tx</code>	RISC-V → UART

5.4.3 Adaptive Threat Scoring

The threat score accumulator adds the weight of each fired alert every clock cycle and subtracts a configurable decay rate every second. The RISC-V polls the score register at one-second intervals and maps the result to one of three severity levels:

- **GREEN** (score < 30): Normal traffic. Rule thresholds remain at their defaults.
- **YELLOW** ($30 \leq \text{score} < 100$): Elevated activity. The RISC-V tightens the TTL threshold and lowers the fragmentation detection sensitivity.
- **RED** (score ≥ 100): Active attack. The RISC-V sets the TTL threshold to its maximum and enables all optional checks.

This feedback loop means the system becomes progressively more sensitive during an attack and automatically relaxes back to baseline once the attack subsides.

5.4.4 UART Alert Output

The RISC-V formats alert data into human-readable strings and transmits them over UART through a memory-mapped transmitter at address 0x800. A typical alert message contains the severity level, the triggered rule, and the source IP address of the offending packet.

5.5 Packet Capture Module (`packet_capture.vhd`)

The packet capture module acts as a Wireshark-like header recorder. On each `frame_done` pulse, it latches all parsed header fields (Ethernet MACs, EtherType, all IPv4 fields, ports, TCP flags, sequence number) into a set of registers accessible from the RISC-V address space. A `capture_mode` bit selects between logging every frame and logging only frames that triggered at least one alert. The RISC-V can read these registers after each alert to reconstruct the offending packet's header for UART reporting.

5.6 Future Hardware Integration Steps

The simulation-verified pipeline is ready for hardware deployment. The remaining steps are:

1. Create a Vivado block design targeting xc7z020clg400-1. Configure the Zynq PS IP for GigE MAC with DMA, add the AXI-Stream DMA IP, and connect the IDS PL design as a custom IP block.
2. Write a minimal PS bare-metal C driver in Vitis that receives frames from the GigE MAC and forwards raw bytes to the PL via the DMA.
3. Apply the XDC pin constraints from Table ??.
4. Validate with Scapy-generated test packets from a PC connected to the board over Ethernet.

If hardware integration is not completed before the defense, the simulation demonstration is an equivalent and fully accepted validation of the IDS functionality.

Chapter 6

Conclusion

6.1 Summary of Achievements

This project produced a fully simulation-verified hardware IDS pipeline comprising eight VHDL modules and twenty detection rules across the Ethernet, IPv4, TCP, and UDP protocol layers. Every rule was verified individually and then validated again in a full end-to-end testbench that exercises the complete chain from raw dibit input to alert latch output.

The integration architecture connects the IDS to a RISC-V co-processor through a 29-register memory-mapped bank. This gives the RISC-V read access to live traffic statistics and alert data, and write access to configurable rule parameters, without requiring any changes to the IDS hardware itself. The resulting system combines the deterministic throughput of hardware inspection with the programmable adaptability of a software processor.

6.2 Academic Contribution

Most academic hardware IDS projects present a fixed set of rules synthesized into an FPGA and verified with pre-recorded packet traces. This project goes further in two respects:

1. **Runtime reconfigurability:** Detection thresholds and forbidden port lists can be updated at runtime through the RISC-V without FPGA reprogramming.
2. **Adaptive severity:** The weighted scoring engine with decay creates a feedback loop between traffic behaviour and rule sensitivity, approximating the behaviour of an adaptive IDS in hardware.

6.3 Limitations

The current system does not inspect packet payloads. Rules are limited to header fields, which means application-layer attacks (such as SQL injection or cross-site scripting carried in HTTP bodies) are outside the scope of detection. Additionally, the hardware integration with the Zynq PS has been designed and planned but not yet validated on physical hardware; the simulation results are the primary evidence of correctness.

6.4 Future Work

The natural extension is deep packet inspection: a `payload_extractor.vhd` module would buffer the TCP or UDP payload into a dual-port BRAM, allowing the RISC-V to read payload bytes and search for known attack signatures. A configurable IP blacklist table stored in BRAM would allow the RISC-V to maintain a dynamic blocklist of known-malicious source addresses. ICMP parsing would add network reconnaissance detection (ping sweeps, ICMP redirect attacks). Finally, completing the hardware bring-up on the Z7-Lite would demonstrate wire-speed detection on a live network segment.

Bibliography

- [1] J. Postel, *Internet Protocol*, IETF RFC 791, September 1981.
- [2] J. Postel, *Transmission Control Protocol*, IETF RFC 793, September 1981.
- [3] J. Postel, *User Datagram Protocol*, IETF RFC 768, August 1980.
- [4] IEEE Std 802.3-2022, *IEEE Standard for Ethernet*, IEEE, 2022.
- [5] Xilinx Inc., *Zynq-7000 SoC Technical Reference Manual*, UG585, v1.13, 2022.
- [6] Xilinx Inc., *AXI DMA LogiCORE IP Product Guide*, PG021, v7.1, 2021.
- [7] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, RISC-V Foundation, 2019.
- [8] G. Lyon, *Nmap Network Scanning*, Insecure.com LLC, 2009.
- [9] M. Roesch, “Snort: Lightweight intrusion detection for networks,” in *Proc. USENIX LISA*, 1999.